From an ISA to state transitions

An ISA specifies the state an instruction may observe and change. A microarchitecture chooses the cycles, functional units, and energy used to implement that contract. This chapter builds a single-cycle subset, a five-stage pipeline, hazard controls, and exception recovery before introducing multiple issue and speculation.

For each operation, identify the source register, destination state, and the cost of an incorrect transition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.1 Design scope

The datapath contains storage, movement, and arithmetic resources. Control derives selectors and write enables from instructions, flags, and requests. PC, registers, and memory hold state.

The subset supports `lw`, `sw`, `add`, `sub`, `and`, `or`, `slt`, `beq`, and `j`. Immediate arithmetic, byte accesses, multiply, floating point, and complete exception support require additional logic.

Pipeline timing questions use a simplified subset without architectural branch delay slots. A wrong-path instruction is squashed. This deliberately differs from the delayed pre-R6 assembly in Chapter 2 and Appendix B. Their flush counts and return rules cannot be substituted here.

Mark every bus width. A MIPS32 word is 32 bits; sequential PC advance is four bytes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An implementable instruction contract

Specify source operands, destination state, memory accesses, target calculation, exception behavior, and adjacent read/write dependencies.

`lw rt, imm(rs)` reads the base, sign-extends the displacement, adds an address, obtains data, and writes `rt`. A following consumer must wait for that data or receive it through an appropriate bypass. An address is not the loaded value.

Correctness concerns final architectural effects rather than a plausible-looking waveform.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2 Clocking and edge-triggered state

Combinational outputs respond after propagation delay. An edge-triggered register samples only at its active edge. A combinational feedback path cannot stand in for several clocked updates.

$$T_{clk} \geq T_{cq} + T_{logic,max} + T_{setup} + T_{skew}.$$

Here clock-to-Q, maximum logic delay, setup requirement, and a conservative adverse clock-skew allowance all have time units. Hold correctness instead constrains the fastest path. A maximum-delay check does not prove hold timing.

Physical timing depends on process, voltage, temperature, wires, and clock distribution. A gate is not intrinsically one cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.3 Datapath building blocks

PC, adders, instruction memory, data memory, a register file, an ALU, and multiplexers form the teaching datapath. The register file provides two reads and one write. Reads are combinational under the chosen timing model; writes occur at an edge.

Separate instruction and data memories permit IF and MEM concurrently. A unified single-port memory would introduce a conflict. Additional ports or different staging are alternative implementations.

The 32 architectural registers include `$zero`, whose reads return zero and whose writes are discarded.

Count accesses and specify read/write timing before deciding whether a structural hazard exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ALU control and main control

Main control generates `RegDst`, `ALUSrc`, `MemtoReg`, `RegWrite`, `MemRead`, `MemWrite`, `Branch`, and `Jump`. ALU control combines the instruction class with the R-format function field.

ALUOp	Class	Operation
00	Load/store	Address addition
01	Branch equal	Subtraction or equality comparison
10	R-format	Function decode

A truth table or ROM can implement control. Its propagation delay also belongs in the timing analysis.

Selecting addition alone does not complete an instruction. The destination selector and write enable must also be correct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

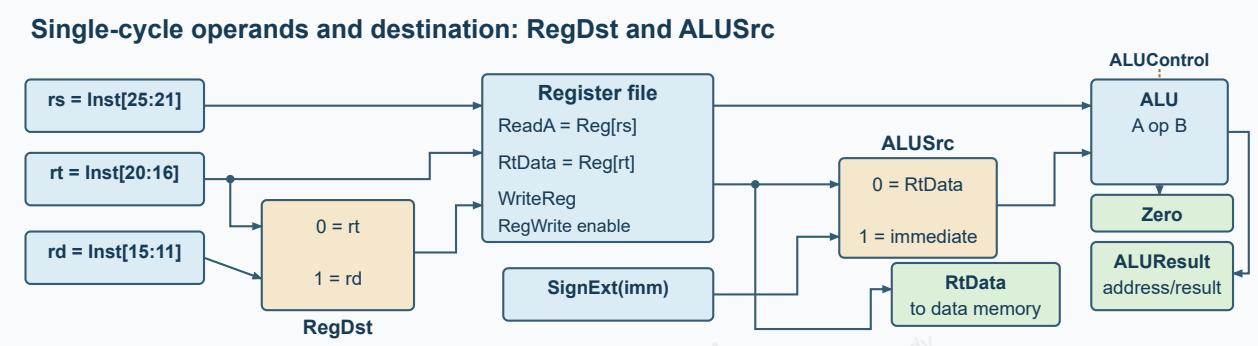
Single-cycle dataflow

This teaching subset has no architectural delay slot. It supports R-type ALU operations, `lw`,`sw`,`beq`,`j`, separate instruction/data memories, combinational reads, and edge-committed PC/register/store writes. It has no multicycle IR/A/B/MDR state or shared-memory IorD selector.

PC.Q addresses instruction memory. Its fields feed decoding and register reads in parallel. Only enabled state updates commit. Instructions do not all traverse a single PC→IM→Reg→ALU→DM→WB chain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

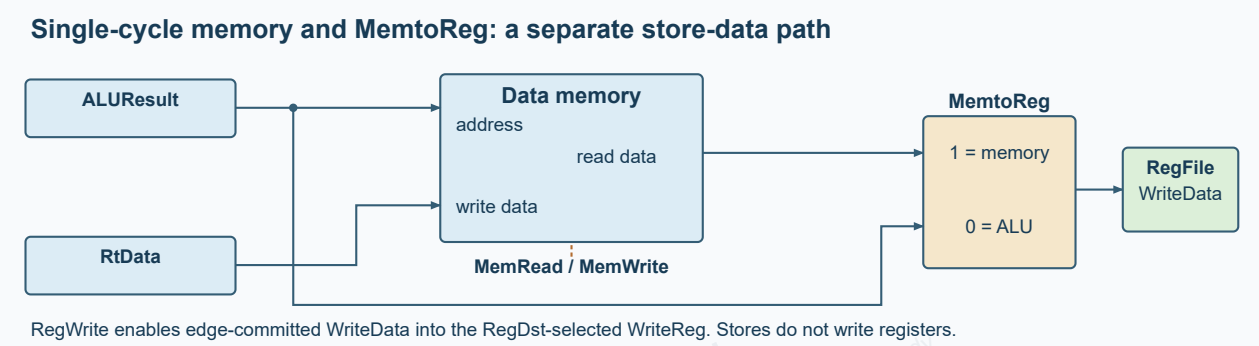
Single-cycle dataflow (continued)



RegDst=0 selects rt at bits 20:16; RegDst=1 selects rd at 15:11. ReadA=Reg[rs] and RtData=Reg[rt]. ALUSrc=0 selects RtData for ALU input B; ALUSrc=1 selects sign-extended immediate. Input A is ReadA. RtData independently supplies store data. Selecting an immediate for address arithmetic must not replace the store value.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Memory and write-back selection

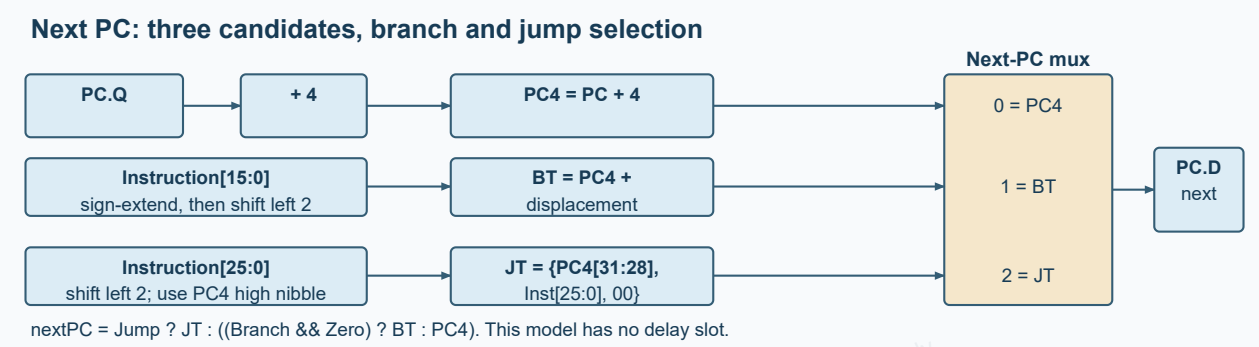


For lw/sw, ALUResult is the effective address; for R-type it is the arithmetic result. MemWrite gates the edge-committed memory write from RtData. MemRead supplies the addressed value to MemtoReg.

MemtoReg=0 selects ALUResult; =1 selects memory data. RegWrite gates WriteData into the RegDst-selected WriteReg. A destination of zero leaves `$zero` unchanged. Identically named endpoints across figures are wires of the same cycle, not extra pipeline registers.

For `sw $t2,-4($s0)`, with s0=0x1000, t2=0x12345678, ALUSrc=1 yields address 0x0ffc, while store data stays 0x12345678. MemWrite=1 and RegWrite=0. Connecting the ALU's immediate input to memory data would incorrectly store -4.

Three next-PC candidates



$$PC4 = (PC + 4) \bmod 2^{32}, \qquad BT = (PC4 + (\text{signext}(imm) \ll 2)) \bmod 2^{32}.$$

A beq subtracts ReadA−RtData; Zero tests equality. Jump target JT concatenates PC4[31:28], instruction[25:0], and 00. It does not use the old PC's high bits.

```
nextPC = Jump ? JT : ((Branch && Zero) ? BT : PC4)
```

Two two-input muxes can select branch/sequential first and jump second. The next edge commits nextPC; this subset executes no delay slot. Every PC4 reference belongs to the same cycle.

Three next-PC candidates (continued)

At PC=0x00400020, immediate=-2, PC4=0x00400024 and BT=0x0040001c. Equal operands select BT; unequal operands select PC4. At PC=0x0ffffffc and jump field zero, PC4=0x10000000 and JT=0x10000000. Using the old PC's high nibble would wrongly produce zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Control table and mux selections

Signal	lw	sw	R-type	beq	j
RegDst	0	X	1	X	X
ALUSrc	1	1	0	0	X
MemtoReg	1	X	0	X	X
RegWrite	1	0	1	0	0
MemRead	1	0	0	0	0
MemWrite	0	1	0	0	0
Branch	0	0	0	1	0
Jump	0	0	0	0	1
ALUOp	00	00	10	01	X

ALUOp 00 selects address addition, 01 subtraction for equality, and 10 funct decoding for add/sub/and/or/slt. X is a don't-care only when its output cannot commit. RegWrite and MemWrite must remain correct. Illegal opcodes and exceptions need additional control outside this subset.

Complete state effects for five instruction classes

Each row independently starts at PC=0x00400020, s0=0x1000, s1=7, s2=5, t2=0x12345678, and Mem[0x100c]=0xabcdef01. Word accesses are aligned and no unsupported overflow occurs.

Instruction	Selected data	State committed at edge
`addu t0,s1,s2`	ReadA=7, RtData=5; ALUSrc 0; ALU=12; MemtoReg 0; RegDst 1	t0=12, PC=0x00400024
`lw t1,12(s0)`	ReadA=0x1000, imm=12; ALU=0x100c; MemtoReg 1; RegDst 0	t1=0xabcdef01, PC=0x00400024
`sw t2,-4(s0)`	ALU=0x0ffc; RtData=0x12345678 goes directly to memory	Mem[0x0ffc]=0x12345678, PC=0x00400024
`beq s1,s2,-2`	ALU=7−5=2, Zero 0, Branch 1, Jump 0	no register/data writes, PC=0x00400024
`j 0x00401000`	field=0x00100400, Jump 1	no register/data writes, PC=0x00401000

Changing only both branch inputs to 5 sets Zero and next PC=0x0040001c. The rt field is a destination for lw and a source for sw/beq.

The slowest path sets the common clock

The load path includes PC clock-to-Q, instruction read, register read, address ALU, memory read, MemtoReg, and setup:

$$T_{clk} \geq T_{PCQ} + T_{IM} + T_{regread} + T_{ALU} + T_{DM} + T_{mux} + T_{setup}.$$

The stated combined delays sum to $30 + 200 + 120 + 180 + 250 + 20 + 40 = 840$ ps. The simplified R-type path is $30 + 200 + 120 + 180 + 40 = 570$ ps but still uses the common 840 ps period. Unlisted wires/control/mux costs must already fit the combined terms; this is not timing closure for a physical chip.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Single-cycle performance and limits

In a separate timing model from the previous 840 ps budget, with period 800 ps and CPI 1, a mix of 60% arithmetic, 25% loads/stores, and 15% control transfers takes

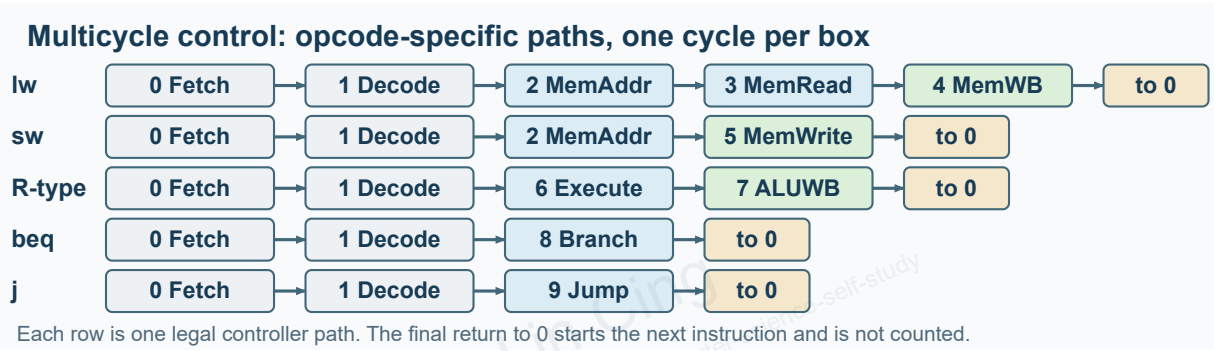
$$T = IC \times 800 \text{ ps}.$$

Shortening a noncritical arithmetic path does not change this time if the load path still determines the clock. Extending the critical load path slows every instruction. Multicycle control and pipelining avoid requiring all instructions to fit one long common period.

A local reduction in logic delay matters only if it changes the limiting path or resource.

Multicycle datapath and control

Internal IR, A, B, ALUOut, MDR retain the instruction, operands, ALU result, and memory data between cycles. PC tracks the architectural instruction sequence.



Each row is one legal controller path. The final return to 0 starts the next instruction and is not counted.

Instruction	State path	Cycles
R-type	Fetch → Decode → Execute → ALUWB → Fetch	4
lw	Fetch → Decode → MemAddr → MemRead → MemWB → Fetch	5
sw	Fetch → Decode → MemAddr → MemWrite → Fetch	4
beq	Fetch → Decode → Branch → Fetch	3
j	Fetch → Decode → Jump → Fetch	3

The last Fetch starts the next instruction. A store never passes through MemRead.

Multicycle datapath and control (continued)

State	Edge effect	Enables and selection
0 Fetch	IR←Memory[oldPC]; PC←oldPC+4	IorD 0, MemRead 1, IRWrite 1, PCWrite 1
1 Decode	A←Reg[rs]; B←Reg[rt]; ALUOut←PC+branch displacement	no architectural register/memory write
2 MemAddr	ALUOut←A+signext(imm)	base/immediate ALU inputs
3 MemRead	MDR←Memory[ALUOut]	IorD 1, MemRead 1
4 MemWB	Reg[rt]←MDR	RegDst 0, MemtoReg 1, RegWrite 1
5 MemWrite	Memory[ALUOut]←B	IorD 1, MemWrite 1
6 Execute	ALUOut←A op B	ALUOp selects funct decoding
7 ALUWB	Reg[rd]←ALUOut	RegDst 1, MemtoReg 0, RegWrite 1
8 Branch	if A=B, PC←saved target	PCWriteCond 1, PCSrc=branch; ALU compares A−B
9 Jump	PC←{currentPC[31:28], IR[25:0], 00}	PCWrite 1, PCSrc=jump

Multicycle datapath and control (continued)

Unlisted architectural enables are zero. Decode dispatches lw/sw→2, R-type→6, beq→8, j→9; other opcodes use an explicit error transition. State 3 goes only to 4, never 5.

Fetch reads Memory[oldPC] and increments PC at the same edge. Decode therefore uses the already incremented PC for its target and must not add another four. For oldPC=0x00400008, imm=-2, fetch leaves PC=0x0040000c and target=0x00400004. A repeated increment wrongly gives 0x00400008. A not-taken branch leaves sequential PC unchanged. Branch/jump selection cannot also assert unconditional sequential update.

For a mix of 45% R-type, 20% branch, 20% load, 15% store,

$$CPI = 0.45(4) + 0.20(3) + 0.20(5) + 0.15(4) = 4.0.$$

$$T = 10^9(4.0)(300\ \text{ps}) = 1.2\ \text{s}.$$

The comparison single-cycle design at 800 ps and CPI 1 takes 0.8 s. Shorter clocks do not guarantee a win when extra cycles outweigh the reduction. Hold the delay budget and mix fixed.

Two FSM invariants are that an instruction never commits RegWrite/MemWrite twice in the same state, and completion retains all necessary operands/results before returning to Fetch. Undefined opcodes must not default to an arbitrary memory write.

4.5 The five-stage pipeline

IF fetches and forms PC+4. ID decodes, reads registers, and forms immediates/control. EX computes ALU results, addresses, or branch equality. MEM accesses data memory. WB writes enabled register results.

Pipeline register	Required saved state
IF/ID	valid, instruction PC, instruction, PC4
ID/EX	valid, PC/PC4, rs/rt/rd, A/B, sign-extended immediate, controls
EX/MEM	valid, PC, ALU result, store data, destination, MemRead/MemWrite/RegWrite/MemtoReg
MEM/WB	valid, PC, memory data or pass-through ALU result, destination, RegWrite/MemtoReg

These are edge-updated snapshots. A destination or RegWrite omitted from EX/MEM cannot be reconstructed at WB. A load's EX result is only an address; its value arrives at the end of MEM.

4.5 The five-stage pipeline (continued)

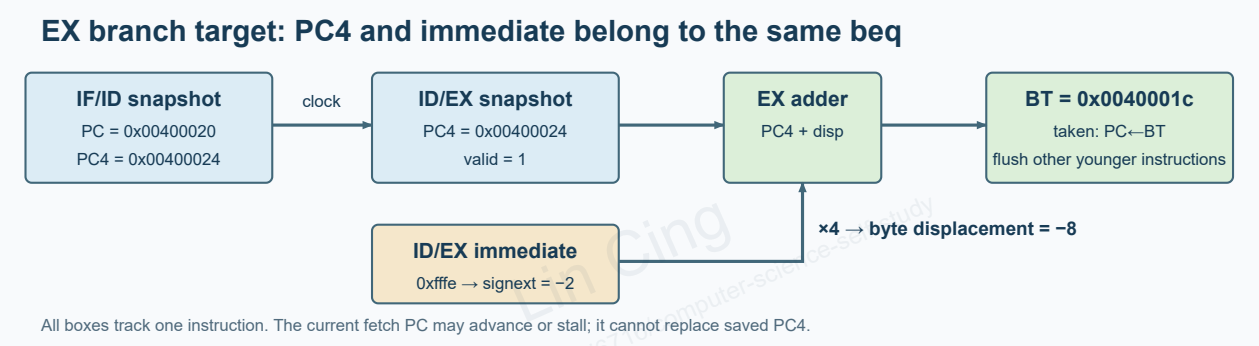
PC/valid, operands, and controls belong to the same instruction and must hold or advance together. Bubble/flush sets valid=0, and valid gates all register/memory writes. A load-use stall holds PC and IF/ID, inserts an invalid ID/EX item, and lets older entries advance. Per-instruction PC identifies the correct EPC for faults. Keeping both PC and PC4 is a teaching choice; PC can also be recovered as PC4−4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The branch target in EX

IF's PC4 travels through IF/ID and ID/EX. EX must not use the global PC, which already fetches younger instructions.

$$BT = (\mathrm{ID/EX.PC4} + 4\,\mathrm{signext}_{16\to32}(imm_{16})) \bmod 2^{32}.$$



A branch at 0x00400020 with bits 0xfffe retains PC4=0x00400024. Signed immediate -2 gives -8 bytes and BT=0x0040001c. Taken selects BT; not-taken keeps the sequential path. A younger stall/flush does not change this saved PC4. Subtracting a fixed amount from the current fetch PC is not equivalent.

Ideal time without hazards

For balanced stage time t and k=5,

$$C_{ideal} = k + (n - 1).$$

A steady-state result completes every cycle but each instruction still takes 5 t. A hundred instructions need 104 cycles. In the long-run model below, each penalty is an actual extra cycle per retired instruction, with no overlap or double counting:

$$CPI_{pipe} = 1 + stall_{loaduse} + mispredict_cycles + structural_bubbles.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ideal pipeline time

For n instructions and k balanced stages, absent hazards and register overhead, completion requires $k + n - 1$ cycles. With five stages and 100 instructions this is 104 cycles.

If the unpipelined instruction takes five equal stage-time units, the time speedup is $500/104 \approx 4.81$. It approaches five for a long stream. Individual instruction latency remains five stages; throughput improves by overlap.

Comparing cycle counts without comparing cycle durations is not a valid physical speedup.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Structural hazards

Two active instructions may need the same exclusive resource. A single memory port cannot simultaneously serve an instruction fetch and a data access.

Register files can permit WB writes and ID reads in one cycle under an explicit write-before-read convention or bypass. Multiple issue increases operand-port demand. Extra ALUs and ports reduce conflicts at area and power cost.

A resource conflict is determined by simultaneous demand, not by the resource's name.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Data dependencies

```
add $8, $9, $10
sub $11, $8, $12
```

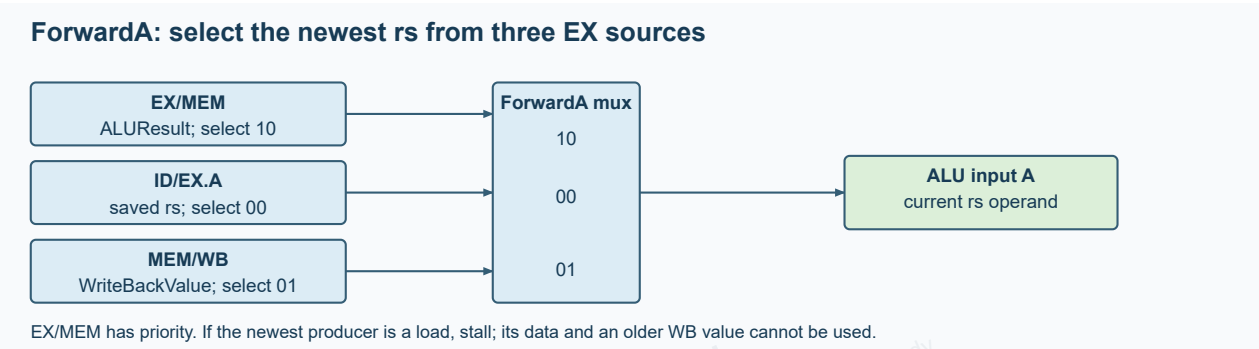
The subtraction has a true read-after-write dependence. Scheduling and forwarding reduce waiting but cannot remove the need for the produced value.

```
add $8, $9, $10
sub $8, $11, $12
```

This pair instead illustrates output-name reuse. Anti-dependencies similarly arise when a later write reuses an earlier source name. Register renaming can remove these false dependencies by assigning different physical registers.

Track the producer of a value rather than treating every repeated architectural name as a true dependence.

Correct forwarding paths



Produced EX/MEM or MEM/WB values can bypass the register file. An ALU consumer immediately after its producer uses EX/MEM. Every comparison checks producer RegWrite, nonzero destination, source use, matching register number, and availability. A load in EX/MEM contains an address rather than loaded data.

For an EX-used source s, define E as `EX/MEM.RegWrite && EX/MEM.rd!=0 && EX/MEM.rd==s` and W as the equivalent MEM/WB match.

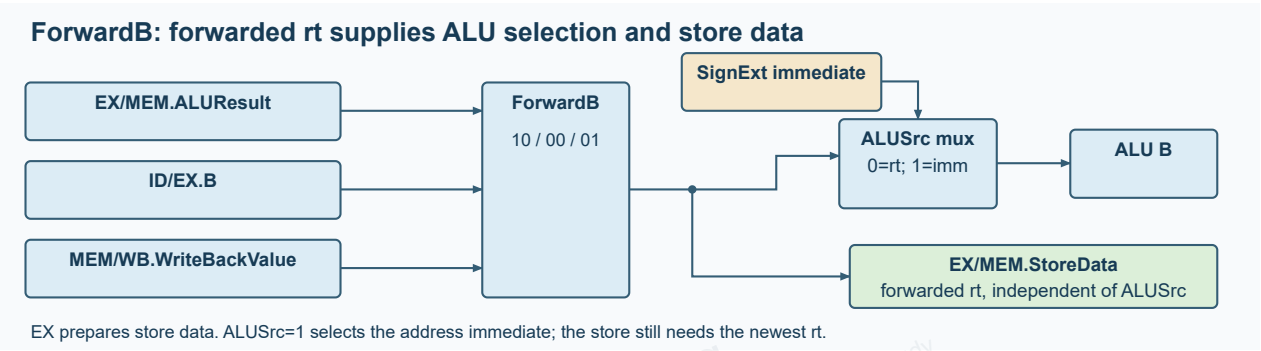
Correct forwarding paths (continued)

Condition	ALU input
E and producer is not load	EX/MEM.ALUResult, code 10
not E and W	MEM/WB.WriteBackValue, code 01
not E and not W	ID/EX saved value, code 00
E and producer is load	stall; neither its address nor an older WB value is correct

ForwardA applies to rs. ForwardB applies to the register operand rt, then ALUSrc chooses that value or the immediate. This model prepares store data in EX; an immediately preceding load therefore conservatively requires a bubble even for a store-data dependence. A separate MEM store-forwarding path can remove it only with stated wiring and conditions.

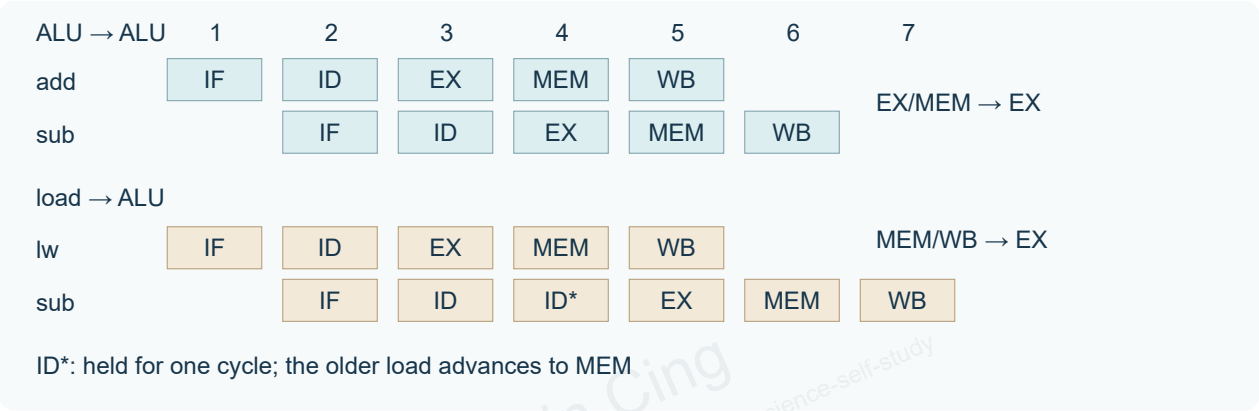
If two producers write s, EX/MEM is newer and wins. In `add $1,...; sub $1,...; and $2,$1,$3`, the AND needs the SUB result.

Store forwarding and ALUSrc



ALUSrc selects only the second ALU operand. A store computes base plus immediate while independently carrying the newest rt data. ForwardB branches to the ALUSrc register input and EX/MEM.StoreData. These cannot be replaced by one common mux selection.

Two dependencies: why does only one stall?



Assume a five-stage single-issue machine, one-cycle memory, EX/MEM and MEM/WB forwarding to EX, and no other hazards. For `add $s0,$t0,$t1; sub $t2,$s0,$t3`, the add result is ready at the end of EX in C3 and forwards from EX/MEM to sub in C4: no stall.

For `lw $t0,4($s0); sub $t2,$t0,$s1`, data is ready only at the end of MEM in C4, too late for EX at the beginning of that cycle. Hold PC and IF/ID for one cycle and insert a bubble; sub uses MEM/WB in EX at C5.

Load-use stalls

```
lw  $2, 20($1)
and $4, $2, $5
```

The load obtains data at MEM's end. The adjacent consumer would otherwise need it in EX in that same cycle. One bubble is required in this model.

Hold PC and IF/ID, clear ID/EX side-effect controls, and let the producer advance. The following cycle forwards the data from MEM/WB. Only genuine source operands should trigger this check. A load's `rt` is a destination, whereas a store's `rt` supplies store data.

Store-data timing can admit a later MEM-stage bypass. State which bypasses exist before declaring a store stall necessary.

For this subset, use_rs/use_rt describe operands needed in EX, including store data prepared there. Both are 1 for R-type/beq/sw; lw uses only rs; j uses neither. A load's destination is ID/EX.rt.

Load-use stalls (continued)

```
stall = ID/EX.MemRead && ID/EX.rt != 0 &&
        ((use_rs && ID/EX.rt == IF/ID.rs) ||
         (use_rt && ID/EX.rt == IF/ID.rt))
PCWrite = !stall
IFIDWrite = !stall
IDEXControls = stall ? all_zero : decoded_controls
```

`lw $2,0($1); lw $2,4($3)` has no dependence through rt because the second load overwrites it. Loading `$zero` creates no register dependency, although its memory access may still fault. Larger ISAs need per-opcode source-use stages rather than treating every encoded field as a read.

A numeric trace: an older value cannot replace an unready one

Use the five-stage, single-issue, in-order model with no cache misses, branches, or structural conflicts. EX/MEM and MEM/WB bypasses feed EX operands. Load data becomes available at the end of MEM for use in the next cycle. WB writes the register file in the first half of a cycle; ID reads it in the second half.

Initially, `$1=0x1000`, `$6=0x30`, `$7=0x03`, `$5=0x0f`, `$9=0x80`, and `Mem[0x1014]=0x3c`. Other relevant registers are zero. Accesses are aligned and no exceptions occur.

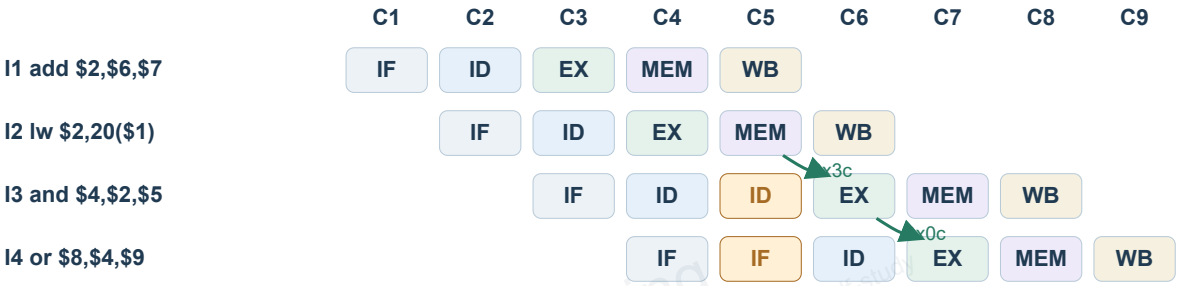
```
I1  add $2,$6,$7     # $2 <- 0x33
I2  lw  $2,20($1)    # $2 <- 0x3c, replacing I1's value
I3  and $4,$2,$5     # $4 <- 0x0c
I4  or  $8,$4,$9     # $8 <- 0x8c
```

I3 needs I2's most recent definition of `$2`, not I1's older value. The RAW chain is `I2 → I3 → I4`. I1 and I2 share a destination, but I2 does not read `$2`, so there is no RAW dependence between them.

Nine cycles: hold the front end, advance older work

IF → ID → EX → MEM → WB

Amber: held instruction. Green arrows: actual forwarded values.



C4: detect dependence → C5: invalid EX bubble; hold I3 in ID and I4 in IF.
C6: MEM/WB → I3.EX. C7: EX/MEM → I4.EX. I4 completes in C9.

Each cell shows the stage occupied during that cycle. Pipeline registers change at the ending edge. C4 detects the dependence between I2.EX and I3.ID. At C4's ending edge, hold PC and IF/ID and inject an invalid ID/EX entry. C5 therefore has no valid EX instruction; the load itself keeps advancing.

Check the values and controls against the timing diagram

Cycle	Instruction positions and available values	Control or result
C4	I2.EX computes address `0x1014`; I3.ID needs `$2`	`PCWrite=0`, `IFIDWrite=0`; inject an invalid ID/EX entry
C5	I1.WB writes `0x33`; I2.MEM obtains `0x3c` at its end; I3 remains in ID	I3 reads and latches old `0x33`; the hazard is now cleared, allowing front-end advance at the edge
C6	I2.WB carries `0x3c`; I3 is in EX	`ForwardA=01` selects MEM/WB, overriding the stale operand; `0x3c AND 0x0f=0x0c`
C7	I3.MEM carries ALU result `0x0c`; I4 is in EX	`ForwardA=10` selects EX/MEM; `0x0c OR 0x80=0x8c`

Without C5's bubble, I3 would reach EX too early. EX/MEM would contain the newest producer's load **address**, `0x1014`, while MEM/WB would hold the older value `0x33`. Neither is the required load result. Using the older value would incorrectly produce `$4=0x03`.

Compare the same program with and without bypasses

Model	I3.EX	I4.EX	Final WB	Extra stall cycles
EX/MEM and MEM/WB forwarding	C6	C7	C9	1
No forwarding; register-file reads only	C7	C10	C12	4

Without forwarding, I3 stalls in ID during C4 and C5, then reads the new `$2` after C6's WB. I4 stalls in ID during C7 and C8, then reads `$4` after C9's WB. Same-cycle write-before-read avoids an additional wait.

Four useful instructions ideally take `4+4=8` cycles. These models take `8+1=9` and `8+4=12` cycles. Their finite-sequence CPIs, `9/4` and `12/4`, include pipeline filling and draining; they are not steady-state CPIs for a long program.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Control hazards, delay slots, and flush

Before a branch reaches EX, younger instructions have entered ID/IF. A taken branch predicted not taken requires recovery. Resolve stage and predictor determine penalty; architectural delay slots are a separate ISA property.

$$mispredict_cycles = f_{branch} \sum_{r} P(wrong_r \mid r) C_r.$$

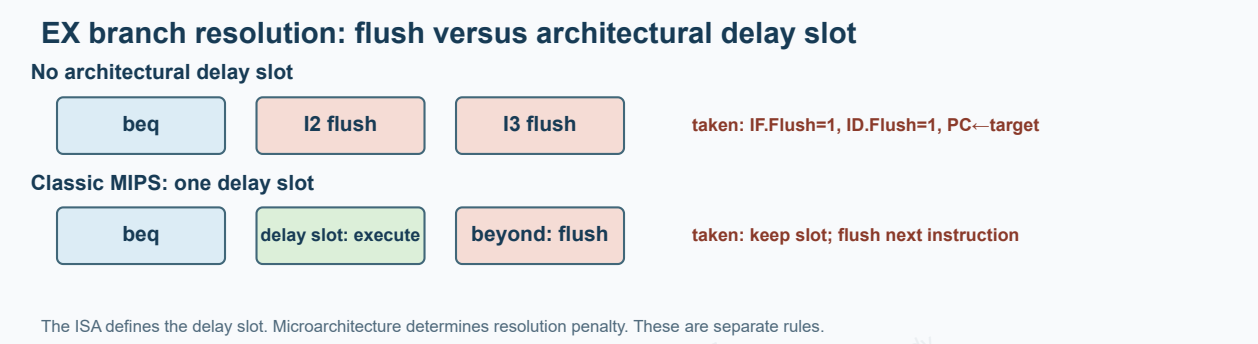
With 20% branches, 60% taken, always-not-taken prediction, and two flushed younger slots,

$$0.20(0.60)(2) = 0.24 \text{ cycles/instruction}.$$

Classic MIPS requires the immediately following slot to execute on either outcome. When the branch is in EX, that slot is in ID and the later instruction in IF.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Control hazards, delay slots, and flush (continued)



The no-slot model flushes ID and IF. The classic model preserves its ID slot and flushes only the later IF instruction. Predict, fetch, resolve, and recover are distinct events. A BTB hit provides a target without guaranteeing correct direction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete CPI boundary

If 10% of instructions additionally require one load-use bubble,

$$CPI = 1 + 0.20(0.60)(2) + 0.10 = 1.34.$$

This includes only the stated penalties. Structural conflicts, cache/target misses, long-latency units, and front-end limits require their own terms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic branch prediction

A two-bit saturating counter has strongly/weakly taken and strongly/weakly not-taken states. Taken outcomes increment toward strongly taken; not-taken outcomes decrement toward strongly not taken. Correctness of the prediction does not alone determine update direction.

Correlating predictors use branch history. Tournament predictors select among mechanisms. A branch-target buffer supplies target information separately from direction prediction. A BTB miss is not necessarily a direction error.

Examine initialization, alternating outcomes, correlation, and table aliasing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.9 Exceptions and interrupts

An exception commonly means a synchronous event caused by an instruction, such as overflow or an illegal opcode. An interrupt commonly means an asynchronous device or timer event. Terminology varies by ISA.

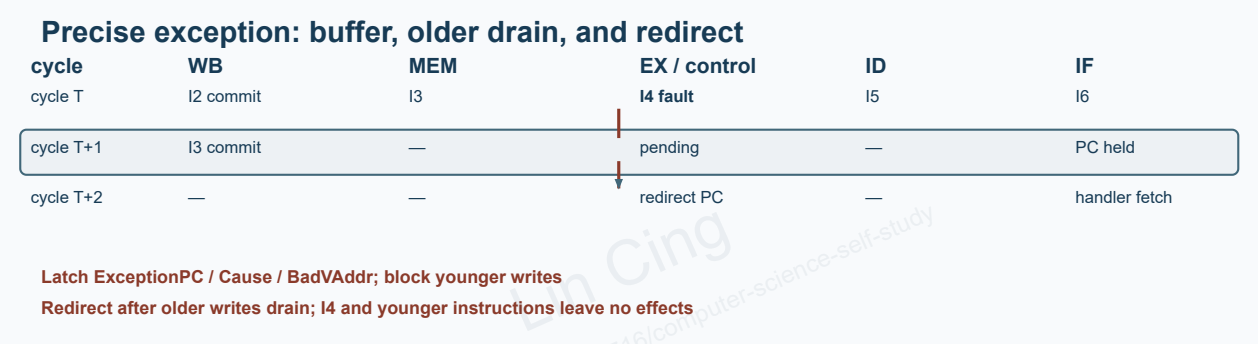
Recovery saves the relevant PC, cause, and address information, transfers to a handler, and either resumes or terminates. The handler needs a state consistent with the architecture rather than an arbitrary mixture of pipeline stages.

A pipeline flush alone does not specify which earlier effects must remain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Precise exception timing and state

Suppose I4 faults in EX. The legal single-issue snapshot has I2 in WB, I3 in MEM, I4 in EX, I5 in ID, I6 in IF; I1 already committed. I2 and I3 have no fault; I3 is a non-store instruction that commits a register result in WB. Prevent architectural writes by I4 and younger entries.



Cycle	WB	MEM	EX/control	ID	IF
T	I2 commits	I3	I4 arithmetic fault	I5	I6
T+1	I3 commits	empty	exception pending	empty	PC held
T+2	empty	empty	PC←handler, saved cause	empty	handler fetch

Precise exception timing and state (continued)

Rows show stage activity during each cycle; commit completes at its end. After I3 commits at the end of T+1, control selects the handler PC at the start of T+2. The first handler instruction is fetched in IF during T+2 and has not yet reached ID.

At T, capture I4.PC, cause, and an address only if required for the exception type; arithmetic overflow need not have a bad address; suppress faulting/younger writes and flush I5/I6. Buffer the exception while fetch stops and older instructions drain. The handler must observe exactly the state after I1, I2, I3 in program order, with no effects from I4 or later. It cannot enter before I3's commit at T+1; this controller redirects at T+2.

Simultaneous exceptions need a specified priority. The oldest synchronous fault commonly wins, while asynchronous interrupts and machine checks have separate policies. Handler entry also saves/switches privilege, status, EPC, cause, and address context.

An out-of-order ROB enforces the same contract. Younger instructions may execute, but retirement remains ordered. Once I4 reaches the head after all older commits, discard I4 and younger entries, restore the committed rename state, and redirect.

4.10 Instruction-level parallelism and multiple issue

ILP is overlap among independent operations in one instruction stream. Pipelining overlaps stages; multiple issue starts several operations in a cycle.

Static issue packets and VLIW expose scheduling decisions to the compiler and binary contract. Dynamic issue selects ready instructions at runtime, tolerating variable latencies at the cost of dependency tracking, wakeup/select, and recovery state.

A four-wide maximum does not imply sustained IPC four.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Superscalar execution and dynamic scheduling

A superscalar front end attempts to supply several instructions each cycle. Issue logic checks operand readiness and functional-unit availability. Later ready work may execute before older stalled work.

Speculation follows an unresolved predicted branch. Recovery discards younger results and restores a checkpoint or rename map. Renaming provides more physical names without increasing the ISA's visible register count.

Out-of-order execution and in-order commit are compatible. Their separation supports precise exceptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A cycle-by-cycle rename, execution, and ROB example

Fix one in-order dispatch and one in-order commit per cycle. Dispatch allocates a ROB entry and a fresh physical destination. Execution starts no earlier than the following cycle. A separate load unit has four-cycle latency; a pipelined integer ALU has one-cycle latency. Results become available at a cycle's end and may feed consumers or commit only in the next cycle. Assume no cache miss, branch, fault, other resource conflict, or ALU overflow.

```
I1: lw  r1,0(r2)
I2: add r3,r1,r4
I3: add r5,r6,r7
I4: add r1,r8,r9
```

Initially architectural r1/r2/... map to p1/p2/.... Rename allocates I1→p11, I2→p12, I3→p13, I4→p14. I2 captures p11 as its r1 source at dispatch. I4 later remaps speculative r1 to p14 but cannot change I2's saved source tag. True RAW dependence I1→I2 remains; distinct physical destinations separate the I1/I4 output name conflict and I2/I4 anti-dependence.

A cycle-by-cycle rename, execution, and ROB example (continued)

Out-of-order results, in-order commit (one commit/cycle)

	1	2	3	4	5	6	7	8	9
I1: lw → p11	D	LD1	LD2	LD3	LD4	C			
I2: add → p12		D	wait	wait	wait	EX	C		
I3: add → p13			D	EX	ready	ready	ready	C	
I4: add → p14				D	EX	ready	ready	ready	C

D: dispatch; EX/LD: execute; wait: operand unavailable; ready: wait for ROB head; C: commit

I2 source = p11 (I1); I4 destination = p14. Result end of cycle → use next cycle.

A cycle-by-cycle rename, execution, and ROB example (continued)

Instruction	Dispatch	Execution	Ready at cycle end	Commit	Reason
I1	1	Load cycles2–5	5	6	Ready ROB head commits next cycle
I2	2	EX cycle6	6	7	Captured p11 waits for I1
I3	3	EX cycle4	4	8	Ready result waits behind I1/I2
I4	4	EX cycle5	5	9	Fresh p14 does not overwrite p11

Take memory[0(r2)]=10, r4=1, r6=2, r7=3, r8=4, and r9=5. I3 produces p13=5 first; I4 produces p14=9; I1 produces p11=10; I2 finally consumes p11 to produce p12=11. Final architectural r1=9, r3=11, r5=5 agrees with program order. Reading p14 for I2 would incorrectly produce r3=10. Executed, ready, and committed are distinct states.

A cycle-by-cycle rename, execution, and ROB example (continued)

In a separate variant, keep I1's loaded value 10 but set r4=2147483647. I2's signed add then reports overflow during EX cycle6. The older I1 may still commit in cycle6. Record I2's fault without committing it. When I2 reaches the ROB head, discard I2/I3/I4 and restore the committed rename map: r1 maps to p11, not p14. The handler sees r1=10 and the pre-sequence architectural r3/r5. Already-computed p13/p14 never committed and cannot survive merely because they finished early.

This model excludes speculative stores, memory disambiguation, and branch checkpoints. Adding them requires a store queue and explicit exception/recovery rules.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Limits to wider issue

If a machine can issue four operations but the program usually exposes only one ready operation, width alone offers little benefit. Average IPC needs a distribution with an explicit denominator, such as instructions issued per observed cycle.

Dependencies, cache bandwidth, prediction, functional-unit mix, register ports, ROB capacity, and power can all limit utilization. An IPC histogram can distinguish these situations better than peak width.

A percentage of cycles and a percentage of instructions produce different weighted averages.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.11 The Opteron X4 case

The book's Barcelona-era Opteron translates x86 instructions into AMD RISC operations, or Rops, before dynamically scheduled speculative execution. The historical model describes up to three Rops issued per cycle and an approximately twelve-stage integer pipeline, with longer floating-point paths.

Complex decoding, prediction misses, dependency chains, cache misses, and translation/rename resources can limit throughput. These figures describe the book's period, not current CPUs.

Keep ISA instruction count separate from internal-operation count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.12 HDL and simulation

An HDL describes ports, combinational equations, and clocked state. A Verilog positive-edge process models a state update. Blocking and nonblocking assignments have different simulation semantics.

Simulation checks the testbench's cases. Synthesis, static timing, formal assertions, and physical testing address other questions. The supplied PDF contains only this section's overview; full HDL is on an unavailable CD. This edition does not claim to reconstruct that missing source.

Assertions about hazards, flushes, and commit order are more informative than a final waveform screenshot alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.13 Processor fallacies

Pipelining primarily improves throughput, not individual instruction latency. Forwarding cannot eliminate a dependence on data that has not yet arrived. Technology affects logic delay, clock distribution, power, and memory access, so pipeline organization is not independent of it.

Give a cycle trace that contradicts each overgeneralization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.14 Checking the complete model

Validate architectural results, enumerate simultaneous resource demands, and specify branch and exception recovery. Report latency, throughput, IPC, frequency, and energy against a workload.

Single-cycle execution exposes critical-path constraints. Pipelining adds overlap together with forwarding, stalls, prediction, and recovery obligations. Multiple issue and dynamic scheduling add more opportunity and more state.

Complete the separate exercises before consulting solutions.

Reconstruct a pipeline diagram from the rules rather than memorizing its appearance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.15 Historical perspective

Multicycle execution distributes a long path over time. Pipelining distributes its stages across different instructions. Both need state to identify unfinished work. Forwarding and stalls maintain value dependencies; prediction trades recovery cost for continuous fetching.

Dynamic scheduling and renaming let ready operations pass stalled ones while retaining true dependencies. Ordered commit preserves the software contract. Wider issue and larger windows increase selection, wiring, energy, and verification costs, so stage count and peak IPC alone cannot rank historical designs.

For each development, identify the constraint removed, the state added, and the state recovery must reconstruct.

References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 4. CSO course slides, processor datapaths and pipelines.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study